Next Greater Element

AlgoExpert · Medium · Stacks

The problem

- For each element of a circular array, the next strictly greater value, or -1.

Intuition

- A monotonic stack holds indices still waiting for a greater value; loop twice for circularity.

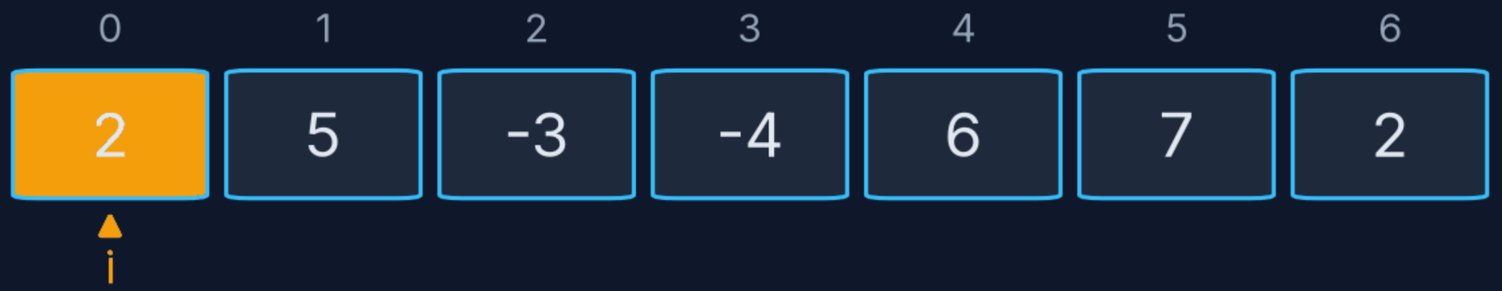
Brute force vs optimal

- For each element scan forward (wrapping) → $O(n^2)$
- Monotonic decreasing stack, two passes for wrap → $O(n)$
- Each index pushed and popped once

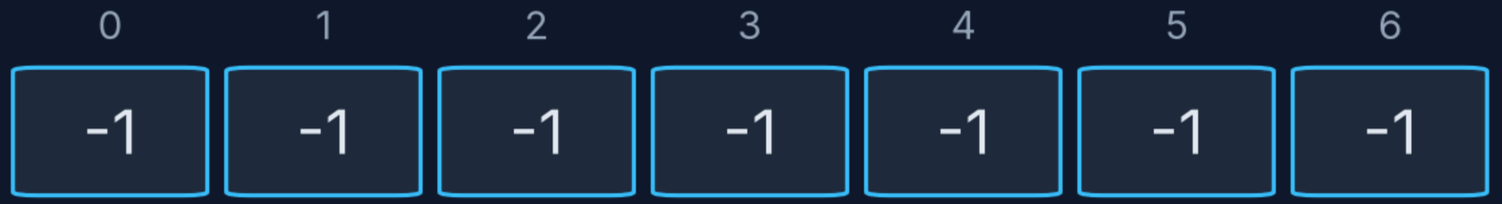
Approach

- 1. Iterate twice around the array
- 2. Pop indices whose value is smaller and assign the answer
- 3. Push the current index

Worked example



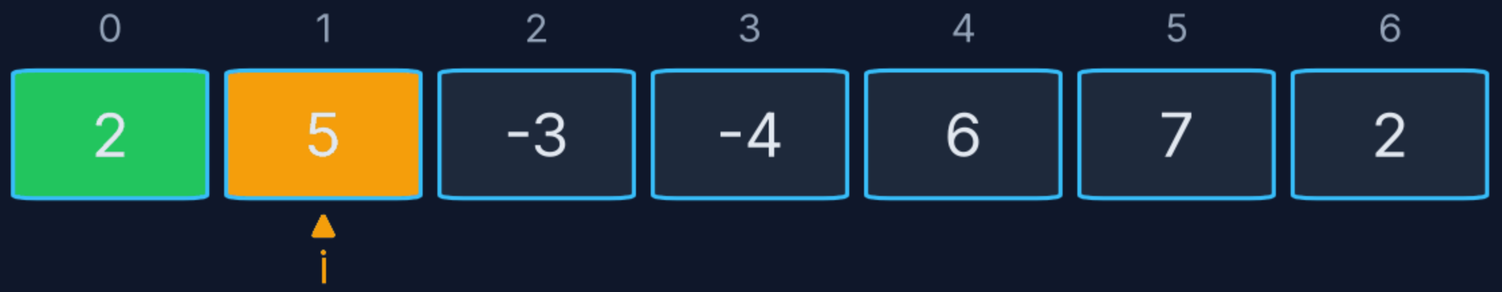
answer



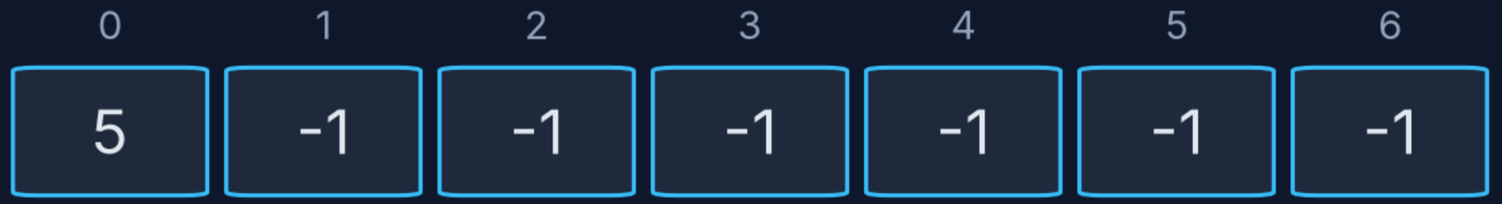

```
stack (indices): [0]
```

Push index 0

Worked example



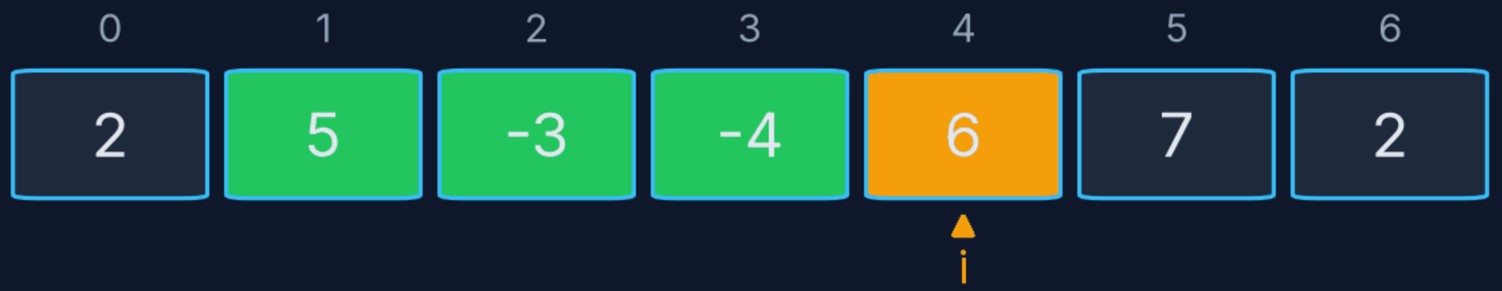
answer




```
stack (indices): [1]
```

5 resolves [2]

Worked example



answer




```
stack (indices): [4]
```

6 resolves [-4, -3, 5]

Worked example



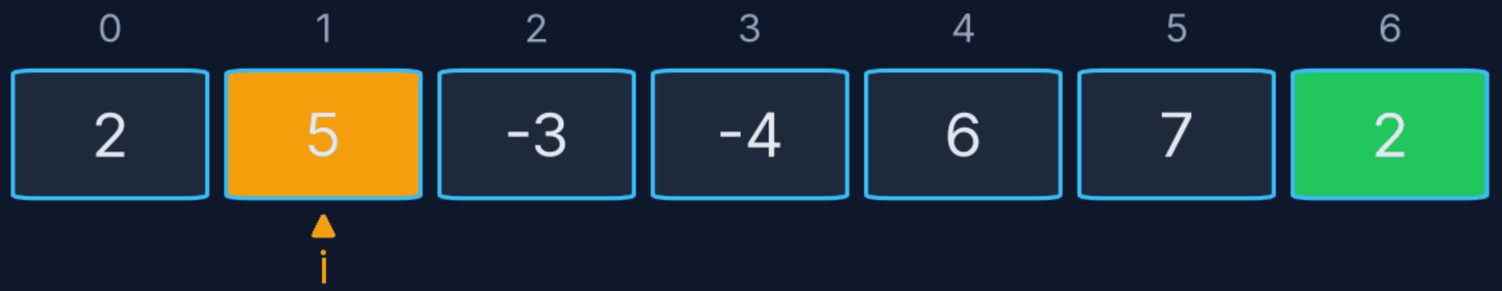
answer




```
stack (indices): [5]
```

7 resolves [6]

Worked example



answer




```
stack (indices): [5]
```

Pass 2: 5 resolves [2]

Java solution

```java
public class NextGreaterElement {
    public static int[] nextGreaterElement(int[] a) {
        int n = a.length;
        int[] out = new int[n];
        Arrays.fill(out, -1);
        ArrayDeque<Integer> st = new ArrayDeque<>();
        for (int i = 0; i < 2 * n; i++) {
            int j = i % n;
            while (!st.isEmpty() && a[st.peek()] < a[j]) out[st.pop()] = a[j];
            if (i < n) st.push(j);
        }
        return out;
    }
}
```

Complexity

- Time: O(n)
- Space: O(n)

Edge cases & pitfalls

- Push indices, not values
- Only push in the first pass
- No greater → −1

Interview tips & follow-ups

- Same pattern: daily temperatures, stock span

Recap

- A monotonic stack holds indices still waiting for a greater value; loop twice for circularity.
- Complexity: O(n)